

Introduction

Behavioral design patterns focus on how objects interact and communicate with each other, helping to define the flow of control in a system. These patterns simplify complex communication logic between objects while promoting loose coupling.

Imagine a TV remote that lets you switch through channels one by one, without needing to know how the channels are stored internally. This kind of controlled access is exactly what behavioral patterns help us achieve.

One such pattern is the **Iterator Pattern**. Let's understand the Iterator Pattern in depth in the upcoming sections.

Iterator Pattern

The **Iterator Pattern** is a behavioral design pattern that provides a way to access the elements of a collection sequentially **without exposing the underlying representation**.

Formal Definition

The Iterator Pattern is a behavioral design pattern that **entrusts the traversal behavior of a collection to a separate design object**. It **traverses the elements without exposing the underlying operations**.

This means whether your collection is an array, a list, a tree, or something custom, you can use an iterator to traverse it in a consistent manner, one element at a time, without worrying about how the data is stored or managed internally.

Real-Life Analogy

Think of a **vending machine**. You don't need to know how the snacks are arranged inside or where exactly your favorite drink is stored. You just press the "**Next**" button to scroll through options one by one. The vending machine controls the order and pace of traversal.

Similarly, an **iterator** acts like that "**Next**" button, giving you one item at a time, hiding the complexity of what's going on behind the scenes.

Understanding the Problem

Let's say we're building a **YouTube Playlist** system. We want to store a list of videos and print their titles one by one. Let's look at the initial code setup:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// A simple Video class
```

```
class Video {
```

```
    string title;
```

```
public:
```

```
    Video(string t) : title(t) {}
```

```
    string getTitle() const {  
        return title;  
    }  
};
```

```
// YouTubePlaylist class
```

```
class YouTubePlaylist {  
    vector<Video> videos;
```

```
public:
```

```
    void addVideo(const Video& video) {  
        videos.push_back(video);  
    }
```

```
    vector<Video>& getVideos() {  
        return videos;  
    }  
};
```

```
// Client Code
```

```
int main() {
```

```
YouTubePlaylist playlist;  
  
playlist.addVideo(Video("LLD Tutorial"));  
  
playlist.addVideo(Video("System Design Basics"));
```

```
// Iterate and print video titles  
  
for (const Video& v : playlist.getVideos()) {  
    cout << v.getTitle() << endl;  
}
```

```
return 0;  
}
```

```
import java.util.*;
```

```
// A simple Video class with title
```

```
class Video {  
  
    String title;  
  
    public Video(String title) {  
        this.title = title;  
    }  
}
```

```
public String getTitle() {  
    return title;  
}  
}
```

// YouTubePlaylist class holds a list of Video objects

```
class YouTubePlaylist {  
    private List<Video> videos = new ArrayList<>();  
  
    // Add a video to the playlist  
    public void addVideo(Video video) {  
        videos.add(video);  
    }  
  
    // Expose the video list  
    public List<Video> getVideos() {  
        return videos;  
    }  
}
```

```
// Client Code
```

```
class Main {  
  
    public static void main(String[] args) {  
  
        YouTubePlaylist playlist = new YouTubePlaylist();  
  
        playlist.addVideo(new Video("LLD Tutorial"));  
  
        playlist.addVideo(new Video("System Design Basics"));  
  
  
        // Loop through videos and print titles  
        for (Video v : playlist.getVideos()) {  
            System.out.println(v.getTitle());  
        }  
    }  
}
```

```
class Video:  
  
    def __init__(self, title):  
        self.title = title  
  
  
    def get_title(self):  
        return self.title
```

```
class YouTubePlaylist:
```

```
    def __init__(self):
```

```
        self.videos = []
```

```
    def add_video(self, video):
```

```
        self.videos.append(video)
```

```
    def get_videos(self):
```

```
        return self.videos
```

```
# Client Code
```

```
if __name__ == "__main__":
```

```
    playlist = YouTubePlaylist()
```

```
    playlist.add_video(Video("LLD Tutorial"))
```

```
    playlist.add_video(Video("System Design Basics"))
```

```
# Iterate and print video titles
```

```
for video in playlist.get_videos():
```

```
    print(video.get_title())
```

What are the Issues?

While the code works, there are several **design-level concerns**:

- **Exposes internal structure:**
 - The internal list or array is directly returned via `getVideos()` or similar methods.
 - This breaks **encapsulation**, as clients can access or even modify the internal collection outside the owning class.
- **Tight coupling with underlying structure:**
 - The external code is tightly bound to the specific type of collection used (like `vector`, `list`, etc.).
 - Any change in the internal structure may require changes in client code.
- **No control over traversal**
 - Traversal logic is managed outside the class.
 - You can't enforce custom traversal behaviors (e.g., reverse, skip elements, filter) without modifying external code.
- **Difficult to support multiple independent traversals:**
 - If two parts of your program want to iterate over the same playlist independently, there's no built-in way to do that cleanly.
 - You have to manage indexing and traversal state manually.

Let us now understand how we can solve this problem using the **Iterator Pattern**.

The Solution

To fix the issues like exposing internal data and lacking control over traversal, we can apply the **Iterator Pattern**. This pattern lets external code access playlist items sequentially without knowing or modifying the

internal data structure.

Let's implement this using custom interfaces and iterator classes.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// ===== Video class representing a single video =====
```

```
class Video {
```

```
    string title;
```

```
public:
```

```
    Video(const string& t) : title(t) {}
```

```
    string getTitle() const {
```

```
        return title;
```

```
    }
```

```
};
```

```
// ===== YouTubePlaylist class (Aggregate) =====
```

```
class YouTubePlaylist {
```

```
    vector<Video> videos;
```

public:

// Method to add video to playlist

void addVideo(const Video& video) {

 videos.push_back(video);

}

// Method to expose internal video list

vector<Video>& getVideos() {

 return videos;

}

};

// ===== Iterator interface =====

class PlaylistIterator {

public:

 virtual bool hasNext() = 0;

 virtual Video* next() = 0;

 virtual ~PlaylistIterator() = default;

};

// ===== Concrete Iterator class =====

```

class YouTubePlaylistIterator : public PlaylistIterator {

    vector<Video>* videos;

    size_t position;

public:

    // Constructor takes the list to iterate on
    YouTubePlaylistIterator(vector<Video>* vids) {

        videos = vids;

        position = 0;

    }

    // Check if more videos are left to iterate
    bool hasNext() override {

        return position < videos->size();

    }

    // Return the next video in sequence
    Video* next() override {

        return hasNext() ? &(*videos)[position++] : nullptr;

    }

};

```

```
// ===== Main method (Client code) =====
```

```
int main() {
```

```
    // Create a playlist and add videos
```

```
    YouTubePlaylist playlist;
```

```
    playlist.addVideo(Video("LLD Tutorial"));
```

```
    playlist.addVideo(Video("System Design Basics"));
```

```
    // Client directly creates the iterator using internal list (not ideal)
```

```
    PlaylistIterator* iterator = new
```

```
    YouTubePlaylistIterator(&playlist.getVideos());
```

```
    // Use the iterator to loop through the playlist
```

```
    while (iterator->hasNext()) {
```

```
        cout << iterator->next()->getTitle() << endl;
```

```
    }
```

```
    delete iterator;
```

```
    return 0;
```

```
}
```

```
import java.util.*;
```

```
// ===== Video class representing a single video =====
```

```
class Video {  
  
    private String title;  
  
    public Video(String title) {  
        this.title = title;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
}
```

```
// ===== YouTubePlaylist class (Aggregate) =====
```

```
class YouTubePlaylist {  
  
    private List<Video> videos = new ArrayList<>();  
  
    // Method to add video to playlist  
    public void addVideo(Video video) {  
        videos.add(video);  
    }  
}
```

```
}
```

```
// Method to expose internal video list
```

```
public List<Video> getVideos() {
```

```
    return videos;
```

```
}
```

```
}
```

```
// ===== Iterator interface =====
```

```
interface PlaylistIterator {
```

```
    boolean hasNext();
```

```
    Video next();
```

```
}
```

```
// ===== Concrete Iterator class =====
```

```
class YouTubePlaylistIterator implements PlaylistIterator {
```

```
    private List<Video> videos;
```

```
    private int position;
```

```
// Constructor takes the list to iterate on
```

```
public YouTubePlaylistIterator(List<Video> videos) {
```

```
    this.videos = videos;

    this.position = 0;
}
```

```
// Check if more videos are left to iterate
```

```
@Override
```

```
public boolean hasNext() {

    return position < videos.size();
}
```

```
// Return the next video in sequence
```

```
@Override
```

```
public Video next() {

    return hasNext() ? videos.get(position++) : null;
}

}
```

```
// ===== Main method (Client code) =====
```

```
public class Main {

    public static void main(String[] args) {

        // Create a playlist and add videos
```

```

YouTubePlaylist playlist = new YouTubePlaylist();

playlist.addVideo(new Video("LLD Tutorial"));

playlist.addVideo(new Video("System Design Basics"));


// Client directly creates the iterator using internal list (not ideal)

PlaylistIterator iterator = new
YouTubePlaylistIterator(playlist.getVideos());


// Use the iterator to loop through the playlist
while (iterator.hasNext()) {

    System.out.println(iterator.next().getTitle());

}

}

}

# ===== Video class representing a single video =====

class Video:

    def __init__(self, title):

        self.title = title


    def get_title(self):

        return self.title

```



```
# ===== YouTubePlaylist class (Aggregate) =====
```

```
class YouTubePlaylist:
```

```
    def __init__(self):
```

```
        self.videos = []
```

```
    # Method to add video to playlist
```

```
    def add_video(self, video):
```

```
        self.videos.append(video)
```

```
    # Method to expose internal video list
```

```
    def get_videos(self):
```

```
        return self.videos
```

```
# ===== Iterator interface =====
```

```
class PlaylistIterator:
```

```
    def has_next(self):
```

```
        raise NotImplementedError
```

```
def next(self):  
  
    raise NotImplementedError
```

```
# ===== Concrete Iterator class =====
```

```
class YouTubePlaylistIterator(PlaylistIterator):
```

```
    def __init__(self, videos):
```

```
        self.videos = videos
```

```
        self.position = 0
```

```
# Check if more videos are left to iterate
```

```
def has_next(self):
```

```
    return self.position < len(self.videos)
```

```
# Return the next video in sequence
```

```
def next(self):
```

```
    if self.has_next():
```

```
        video = self.videos[self.position]
```

```
        self.position += 1
```

```
        return video
```

```
    return None
```

```
# ===== Main method (Client code) =====

if __name__ == "__main__":

    # Create a playlist and add videos

    playlist = YouTubePlaylist()

    playlist.add_video(Video("LLD Tutorial"))

    playlist.add_video(Video("System Design Basics"))


    # Client directly creates the iterator using internal list (not ideal)

    iterator = YouTubePlaylistIterator(playlist.get_videos())


    # Use the iterator to loop through the playlist

    while iterator.has_next():

        print(iterator.next().get_title())
```

How This Solves the Problem

With the iterator pattern in place, we've clearly **separated the concern of how elements are traversed** from the actual data structure that stores them. Here's how this improves our design:

Problem	How Iterator Pattern Solves It
---------	--------------------------------

Direct access to internal data structure	The collection no longer exposes its internal data (like a list or array) directly for traversal. Instead, an iterator is used to access elements one-by-one, encapsulating the structure.
No standard way to iterate	All traversal is now handled through a consistent interface (hasNext() / next()), regardless of how the data is stored internally. This ensures uniformity in how iteration happens.
Traversal logic spread across client code	The logic for maintaining iteration state (e.g., index or position) is encapsulated within the iterator class itself, keeping the client code clean and focused only on usage.
Difficult to customize traversal	Custom iterator classes can easily be extended to provide different traversal strategies (e.g., reverse, filtering, skipping), without changing the underlying collection.
Tight coupling to collection type	Client code no longer depends on the exact type of data structure (like array, list, vector). It interacts only with the iterator, reducing dependencies and improving flexibility.

One Major Issue Still Remains...

Even though we've abstracted the traversal logic into an iterator class, the **client is still responsible for creating and using the iterator**, which is not ideal. The goal of true encapsulation would be to hide even the creation of the iterator, something we'll address now with a more refined approach in the next section.

More Refined Approach

This version fully aligns with the **Iterator Design Pattern**, where the **collection itself provides the iterator**, and the client is **decoupled from the internal list structure**.

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// ===== Video class representing a single video =====
```

```
class Video {
```

```
    string title;
```

```
public:
```

```
    Video(const string& t) : title(t) {}
```

```
    string getTitle() const {
```

```
        return title;
```

```
}
```

```
};
```

```
// ===== Playlist interface =====
```

```
// (acts as a contract for collections that are iterable)
```

```
class PlaylistIterator;
```

```
class Playlist {
```

```
public:
```

```
    virtual PlaylistIterator* createIterator() = 0;
```

```
    virtual ~Playlist() = default;
```

```
};
```

```
// ===== Iterator interface (defines traversal contract)  
=====
```

```
class PlaylistIterator {
```

```
public:
```

```
    virtual bool hasNext() = 0;    // Checks if more elements are left
```

```
    virtual Video* next() = 0;    // Returns the next element
```

```
    virtual ~PlaylistIterator() = default;
```

```
};
```

```
// ===== Concrete Iterator class =====
```

```
// Implements the actual logic for traversing the YouTubePlaylist
```

```
class YouTubePlaylistIterator : public PlaylistIterator {
```

```
    vector<Video>* videos;
```

```
    size_t position;
```

```
public:
```

```
    // Constructor takes the collection to iterate over
```

```
    YouTubePlaylistIterator(vector<Video>* vids) {
```

```
        videos = vids;
```

```
        position = 0;
```

```
    }
```

```
    // Check if more videos are left
```

```
    bool hasNext() override {
```

```
        return position < videos->size();
```

```
    }
```

```
    // Return the next video in the playlist
```

```
    Video* next() override {
```

```
        return hasNext() ? &(*videos)[position++] : nullptr;
```

```
    }
```

```
};
```

```
// ===== YouTubePlaylist class (Aggregate) =====
```

```
// Implements Playlist to guarantee it provides an iterator
```

```
class YouTubePlaylist : public Playlist {
```

```
    vector<Video> videos;
```

```
public:
```

```
    // Method to add a video to the playlist
```

```
    void addVideo(const Video& video) {
```

```
        videos.push_back(video);
```

```
    }
```

```
    // Instead of exposing the list, return an iterator
```

```
    PlaylistIterator* createIterator() override {
```

```
        return new YouTubePlaylistIterator(&videos);
```

```
    }
```

```
};
```

```
// ===== Main method (Client code) =====
```

```
int main() {
```



```
// Create a playlist and add videos to it
```

```
YouTubePlaylist playlist;
```

```
playlist.addVideo(Video("LLD Tutorial"));
```

```
playlist.addVideo(Video("System Design Basics"));
```

```
// Client simply asks for an iterator — no access to internal data  
structure
```

```
PlaylistIterator* iterator = playlist.createIterator();
```

```
// Iterate through the playlist using the provided interface
```

```
while (iterator->hasNext()) {
```

```
    cout << iterator->next()->getTitle() << endl;
```

```
}
```

```
delete iterator;
```

```
return 0;
```

```
}
```

```
import java.util.*;
```

```
// ===== Video class representing a single video =====
```

```
class Video {
```

```
private String title;
```

```
public Video(String title) {
```

```
    this.title = title;
```

```
}
```

```
public String getTitle() {
```

```
    return title;
```

```
}
```

```
}
```

```
// ===== Playlist interface =====
```

```
// (acts as a contract for collections that are iterable)
```

```
interface Playlist {
```

```
    // Method to return an iterator for the collection
```

```
    PlaylistIterator createIterator();
```

```
}
```

```
// ===== YouTubePlaylist class (Aggregate) =====
```

```
// Implements Playlist to guarantee it provides an iterator
```

```
class YouTubePlaylist implements Playlist {
```

```
private List<Video> videos = new ArrayList<>();
```

```
// Method to add a video to the playlist
```

```
public void addVideo(Video video) {
```

```
    videos.add(video);
```

```
}
```

```
// Instead of exposing the list, return an iterator
```

```
@Override
```

```
public PlaylistIterator createIterator() {
```

```
    return new YouTubePlaylistIterator(videos);
```

```
}
```

```
}
```

```
// ===== Iterator interface (defines traversal contract)
```

```
=====
```

```
interface PlaylistIterator {
```

```
    boolean hasNext(); // Checks if more elements are left
```

```
    Video next(); // Returns the next element
```

```
}
```

```
// ===== Concrete Iterator class =====
```

```
// Implements the actual logic for traversing the YouTubePlaylist
```

```
class YouTubePlaylistIterator implements PlaylistIterator {
```

```
    private List<Video> videos;
```

```
    private int position;
```

```
// Constructor takes the collection to iterate over
```

```
public YouTubePlaylistIterator(List<Video> videos) {
```

```
    this.videos = videos;
```

```
    this.position = 0;
```

```
}
```

```
// Check if more videos are left
```

```
@Override
```

```
public boolean hasNext() {
```

```
    return position < videos.size();
```

```
}
```

```
// Return the next video in the playlist
```

```
@Override
```

```
public Video next() {
```

```
    return hasNext() ? videos.get(position++) : null;
```

```

    }
}

// ===== Main method (Client code) =====

public class Main {

    public static void main(String[] args) {

        // Create a playlist and add videos to it

        YouTubePlaylist playlist = new YouTubePlaylist();

        playlist.addVideo(new Video("LLD Tutorial"));

        playlist.addVideo(new Video("System Design Basics"));


        // Client simply asks for an iterator — no access to internal data
        structure

        PlaylistIterator iterator = playlist.createIterator();


        // Iterate through the playlist using the provided interface

        while (iterator.hasNext()) {

            System.out.println(iterator.next().getTitle());

        }

    }

}

```

```
# ===== Video class representing a single video =====
```

```
class Video:
```

```
    def __init__(self, title):
```

```
        self.title = title
```

```
    def get_title(self):
```

```
        return self.title
```

```
# ===== Playlist interface =====
```

```
# (acts as a contract for collections that are iterable)
```

```
class Playlist:
```

```
    def create_iterator(self):
```

```
        raise NotImplementedError
```

```
# ===== Iterator interface (defines traversal contract)
```

```
=====
```

```
class PlaylistIterator:
```

```
    def has_next(self):
```

```
        raise NotImplementedError
```

```
def next(self):  
  
    raise NotImplementedError
```

```
# ===== Concrete Iterator class =====  
  
# Implements the actual logic for traversing the YouTubePlaylist  
  
class YouTubePlaylistIterator(PlaylistIterator):  
  
    def __init__(self, videos):  
  
        self.videos = videos  
  
        self.position = 0  
  
  
    # Check if more videos are left  
  
    def has_next(self):  
  
        return self.position < len(self.videos)  
  
  
    # Return the next video in the playlist  
  
    def next(self):  
  
        if self.has_next():  
  
            video = self.videos[self.position]  
  
            self.position += 1  
  
            return video
```

```
return None
```

```
# ===== YouTubePlaylist class (Aggregate) =====
```

```
# Implements Playlist to guarantee it provides an iterator
```

```
class YouTubePlaylist(Playlist):
```

```
    def __init__(self):
```

```
        self.videos = []
```

```
# Method to add a video to the playlist
```

```
def add_video(self, video):
```

```
    self.videos.append(video)
```

```
# Instead of exposing the list, return an iterator
```

```
def create_iterator(self):
```

```
    return YouTubePlaylistIterator(self.videos)
```

```
# ===== Main method (Client code) =====
```

```
if __name__ == "__main__":
```

```
    # Create a playlist and add videos to it
```



```
playlist = YouTubePlaylist()

playlist.add_video(Video("LLD Tutorial"))

playlist.add_video(Video("System Design Basics"))
```

Client simply asks for an iterator — no access to internal data structure

```
iterator = playlist.create_iterator()
```

Iterate through the playlist using the provided interface

```
while iterator.has_next():

    print(iterator.next().get_title())
```

Key Improvements

- The **YouTubePlaylist** class no longer exposes its internal implementation of **Videos**.
- The client **does not manage or know about the internal structure**.
- The **Playlist** interface allows us to make other types of playlists (e.g., **MusicPlaylist**) that can also be iterable.
- Fully aligns with the **Iterator Design Pattern** principles.

Ideal Scenarios for Using the Iterator Pattern

The Iterator Pattern isn't meant for every situation, but it becomes incredibly useful in specific cases. Here are the key situations where this pattern shines:

- **You want to traverse a collection without exposing its internal structure:**

Instead of revealing whether it's an ArrayList, Vector, or a custom tree, the pattern lets clients access elements one-by-one, safely and uniformly.

- **You need multiple ways to traverse a collection:**

For example, forward traversal, reverse traversal, or skipping every second element. Each of these can be handled by a different iterator implementation without changing the collection itself.

- **You want a unified way to traverse different types of collections:**

Whether it's a list of videos, a set of songs, or a stack of documents, clients should be able to iterate over them using a common interface.

- **You want to decouple iteration logic from collection logic:**

By separating how elements are stored from how they're accessed, you reduce complexity and improve maintainability. Changes in iteration logic won't affect how the collection is structured, and vice versa.

Real World Examples

The Iterator Pattern is deeply embedded in software systems where data needs to be traversed without exposing its internal structure. Here are two crisp, real-world examples:

1. Java Collection Framework

In Java, every collection class, like **ArrayList**, **HashSet**, **TreeSet**, implements the **Iterable** interface, which returns an **Iterator** via the **iterator()** method:

```
List<String> fruits = new ArrayList<>();
```

```
fruits.add("Apple");
```

```
fruits.add("Banana");
```

```
Iterator<String> iterator = fruits.iterator();
```

```
while (iterator.hasNext()) {
```

```
    System.out.println(iterator.next());
```

```
}
```

The client doesn't need to know how the list is implemented internally, just how to get the next element.

2. Python's `iter()` and `next()` Functions

In Python, you can turn any iterable (like a list or tuple) into an iterator using `iter()`, and manually traverse it using `next()`:

```
nums = [10, 20, 30]
```

```
it = iter(nums)
```

```
print(next(it)) # 10
```

```
print(next(it)) # 20
```

Under the hood, this is Python's version of the **Iterator Pattern**. It works the same way across lists, sets, file streams, and even custom objects if you implement `__iter__` and `__next__`.

Pros and Cons

Pros of Iterator Pattern

- **Hides internal structure:**
You can traverse a collection without knowing how it's built internally.
 - **Unified way to traverse:**
You use the same methods (`hasNext`, `next`) regardless of the collection type.
 - **Supports multiple traversal strategies:**
You can easily create different iterators (e.g., forward, reverse, filtered).
 - **Follows SRP and OCP principles:**
Iteration logic is separated (Single Responsibility), and new iterators can be added without modifying existing code (Open/Closed).
-

Cons of Iterator Pattern

- **Adds extra classes/interfaces:**
Requires more boilerplate code to set up custom iterators.
- **Can be overkill for simple data structures:**
For small lists, a direct `for` loop might be more straightforward.
- **External iteration is manual:**
Client has to manage the loop using `hasNext()` and `next()` unless abstracted further.

Class Diagram

TUF

